

AXIOM
SPACE

INTELLIGENT LEARNING AGENT SYSTEM

09

DOC 09

2026 年软件杯 · A3 组

团队级 AI Coding 方法论 与 AI 辅助开发流程说明

Team-level AI Coding Methodology & Development Workflow

完整呈现团队研究并实践的 Rules、Specification、Skills、Loop 与 Harness 方法论。

项目：AXIOM Space

参赛队伍：给春稗以秋实

文档编号：09

版本：提交版 · 2026.07

DOCUMENT 09 / REVIEWER GUIDE

文档导读

本册职责 完整呈现团队研究并实践的 Rules、Specification、Skills、Loop 与 Harness 方法论。

文档信息

文档编号：09
文档性质：团队级 AI Coding 方法论、工程流程与工具使用说明
方法论真源：团队内部研究成果《团队级 AI Coding 简明手册 v0.2》（2026.06，共 30 页）
核心职责：完整说明团队如何用 Rules、Specification、Skills、Loop Engineering 与 Harness Engineering 组织 AI 软件开发，以及这套方法在 AXIOM Space 中如何被实践和验证
关联文档：[03-系统设计与开发说明书](#)、[04-测试说明书与测试报告](#)、[08-开源组件使用与合规说明](#)

目录

Word 中可点击条目直接跳转；目录层级与正文书签保持一致。

01 方法论结论

02 1. 第一性原理与总体设计

- 1.1 根问题：怎样治理非确定性
- 1.2 四个不可互相替代的部件
- 1.3 用具体问题描述处境
- 1.4 软件开发实现地图
- 1.5 软件开发工具体系

03 2. 需求衔接：把需求变成 AI 可执行规格

- 2.1 需求规格的四元结构
- 2.2 原型与设计系统
- 2.3 从需求到测试的无损传递
- 2.4 AXIOM Space 的实践

04 3. 开发实现：让 AI 在项目约束内持续工作

- 3.1 Rules、Specification、Skills
- 3.2 AI 上下文体系
- 3.3 Plan/Go 规格流转
- 3.4 外部工具的三种接入方式
- 3.5 AI 编程工具分层
- 3.6 SDD 框架比较
- 3.7 Superpower 的七段工程顺序
- 3.8 技术规格采用可执行 DSL
- 3.9 打样工程：让 AI 有正确作业可抄
- 3.10 多任务同步开发
- 3.11 研发自测核心 Loop

05 4. 测试验收：用反馈闭环控制 AI 质量

- 4.1 AI 辅助下的分层测试策略
- 4.2 浏览器工具选择
- 4.3 从测试用例生成 E2E 的三段法
- 4.4 Playwright 脚本的最小结构
- 4.5 E2E 超级 Loop
- 4.6 AXIOM Space 的浏览器双轨实践

06 5. Agent as Code：把 AI 工程能力纳入仓库

- 5.1 工作定义
- 5.2 多工具共享的文件设计
- 5.3 AXIOM Space 的实现方式

07 6. 团队实践验证与最终方法论

- 6.1 我们具体设计了什么
- 6.2 我们具体检验了什么
- 6.3 实践中得到的七条方法论
- 6.4 哪些内容是已采用、选型参考和按需启用

- 6.5 最终工作法

08 7. AXIOM Space 的落地旁证

- 7.1 开发阶段 AI 工具
- 7.2 产品运行时 AI 能力
- 7.3 项目文档与方法论的对应

09 8. 30 页 PPT 逐页研究与执行说明

- 8.1 第 1—5 页：方法论全景
- 8.2 第 6—19 页：需求衔接与开发实现
- 8.3 第 20—27 页：测试验收与 Agent as Code
- 8.4 第 28—30 页：研究边界与最终判断

10 9. 参考资料与采用边界

- 9.1 手册列出的研究入口
- 9.2 采用边界

11 结语

方法论结论

AXIOM Space 的 AI Coding 方法不是“让 AI 多写代码”，而是把一个非确定性生成器纳入确定的软件工程：

Rules 管行为边界，**Specification** 管目标事实，**Skills** 管可复用能力，**Loop** 管失败后的持续收敛，**Harness** 管上下文、工具、权限、测试和证据的整体运行环境。

团队对 AI 编程思想进行了五阶段研究，并将项目工作方式推进到 Harness Engineering：

阶段	核心变化	仍未解决的问题	AXIOM Space 的处理
原始 AI 编程	从问答或 AI IDE 获得代码	上下文靠人重复输入，结果不可复现	不作为团队工程方式
Rule 约束	明确能做什么、不能做什么	能约束行为，不能定义本轮目标和完成条件	用 AGENTS.md、CLAUDE.md、目录规范和权限配置固化
SDD	将需求和技术设计结构化为 Spec	有规格仍可能在首次生成后过早停止	建立商业→研究→MVP→PRD→DDD→TDD→测试计划的规格链
Loop Engineering	让测试失败自动返回实现环节	循环仍需要稳定上下文、工具和权限	建立核心研发自测 Loop 与 E2E 超级 Loop
Harness Engineering	将 AI 纳入项目治理体系	—	将规则、规格、脚本、模型工具、测试、浏览器和证据统一放入仓库治理

这套方法形成了四项可复核成果：

- 需求可以无损传递：** 从用户问题、字段、原型和规则，一直传到领域对象、接口、测试和页面。
- AI 可以持续执行：** Plan 锁定上下文，测试提供反馈，失败返回循环，满足准出条件后才退出。
- 工具可以按责任组合：** MCP 负责标准连接，Skills 负责复用能力，Commands 负责直接执行，Browser Use 与 Playwright 分别承担探索和固化。
- 经验可以成为团队资产：** 项目规则、规格、打样代码、测试用例、失败记录和证据随仓库版本化，不随一次对话消失。

AXIOM Space 已经形成对应的工程证据：

方法资产	仓库中的事实
SDD 规格链	docs/01-SDD-文档驱动开发流程/ 下 8 个连续阶段文档和《开发技术规范》
决策过程	docs/02-决策与开发过程记录/ 下 10 个专题决策记录
项目宪法	AGENTS.md、CLAUDE.md
AI 工具配置	.codex/config.toml、.claude/settings.local.json
可执行规格	289 条稳定编号 SDD 用例，覆盖主链路、对象、服务、事件、Agent、UI 和优先级
机器反馈	Node 验收、数据库/API/Agent/Sidecar/UI 测试、Playwright E2E

方法资产	仓库中的事实
浏览器探索	scripts/a3_browser_use.py, 包含场景任务、页面安全边界、PASS/PARTIAL/FAIL 和证据输出
固化回归	test/e2e/sdd-ui.spec.ts 与 playwright.config.ts

1. 第一性原理与总体设计

1.1 根问题：怎样治理非确定性

软件工程要求可重复、可追溯、可验收；大模型输出具有概率性。同一任务在上下文、模型版本、工具状态或表达方式变化时，可能得到不同结果。因此，团队级 AI Coding 的根问题不是“模型会不会写代码”，而是：

怎样让一个非确定性生成器，在确定的业务目标、工程边界和准出标准内持续工作，直到形成可接受结果。

这要求同时控制六个变量：

变量	必须被固定的内容
目标	服务谁、解决什么、成功是什么
上下文	当前架构、对象、接口、历史决定和现有代码
行为	允许修改什么、禁止越过什么、必须遵守什么
能力	可以调用哪些工具、脚本、模型和外部系统
反馈	哪些测试、浏览器结果和人工检查决定继续或退出
责任	谁批准计划、谁接受变更、谁承担最终提交责任

任何一项缺失，AI 的速度都可能转化为返工；六项同时成立，AI 才从个人效率工具升级为团队工程能力。

1.2 四个不可互相替代的部件

部件	工作定义	典型内容	单独使用的局限
Rules	约束 AI 行为的边界	架构、命名、目录、禁止事项、权限	不知道本轮究竟要实现什么
Specification	将模糊要求收敛为事实	需求、字段、领域模型、API、验收标准	不负责实际执行，也不能保证完成
Skills	将提示词、脚本、模板和工具封装为能力	测试、部署、检索、生成、审查	不能自行决定产品目标和授权范围
Loop	让输出接受反馈并反复修复	RED/GREEN、API 测试、浏览器调试、E2E	没有 Harness 时，每次仍要重新搭环境

Harness 位于四者之上：它不是另一个大模型，也不是一个万能插件，而是把 Rules、Specification、Skills、Loop、权限和证据组织成一个项目专用运行环境。

五阶段中的典型实践也有明确差异：

- 原始阶段依赖从浏览器复制代码，或通过问答、AI IDE 反复交互；
- Rule 阶段以 RIPER-5 等方式规定模型在不同工作模式下怎样处理任务，但需求仍主要靠持续对话输入；
- SDD 阶段以 OpenSpec 等规格流转方式实现“文件进、文件出”，需求不再只存在于聊天；
- Loop Engineering 为任务建立验收规则和测试套件，让输出失败后自动返回循环；
- Harness Engineering 进一步为 AI 提供项目约束体系和外部接口，使合格代码成为工程环境的产物，而不是一次偶然生成。

1.3 用具体问题描述处境

手册将团队面临的问题拆成四个问题域。这个分解先确定工程责任，再选择工具。

问题域	需要解决的问题	必须形成的产物
需求衔接	AI 怎样理解新需求、存量修改、字段、原型和规则	需求模板、字段表、原型映射、可测试规则
开发实现	AI 怎样获得上下文、调用工具、保持代码一致并并行工作	规则、规格、Skills、打样工程、Plan
测试验收	AI 怎样从规格生成测试、操作浏览器、判断失败并回归	分层测试、Browser Use、Playwright、Bug 证据
Agent as Code	AI 协作文件怎样进入仓库并被多工具共享	项目宪法、文档真源、AI 目录、配置与权限

1.4 软件开发实现地图

手册设计的不是“编码阶段使用 AI”，而是覆盖需求分析、技术方案、编码、研发自测和 QA 测试的完整地图。

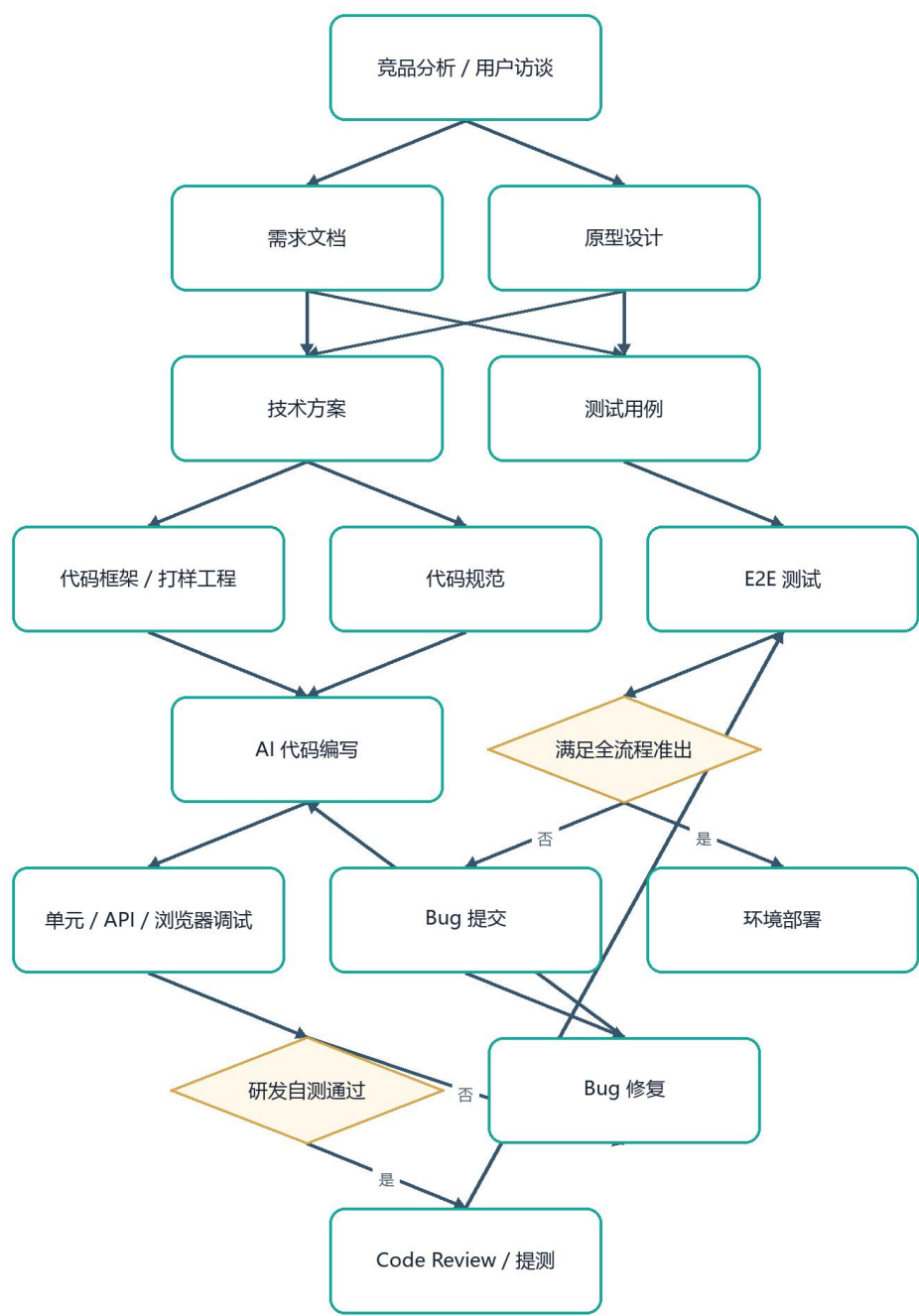


图 1 1.4 软件开发实现地图 (流程/架构图)

图中存在两个不同尺度的循环：

- 5. 研发自测核心 Loop： AI 编码、单元/API 测试、浏览器调试、Bug 修复之间的快速循环；
- 6. 全流程超级 Loop： 从需求、原型、测试用例，到 Code Review、E2E、Bug 回流和部署的端到端循环。

1.5 软件开发工具体系

AI 不直接从一句自然语言跳到部署。每一阶段都应有输入模板、结构化中间产物和可检查输出。

阶段	输入资产	AI 参与	输出资产
需求	需求规格模板、研究材料	整理、切模块、补字段、正交化规则	需求规格 Markdown
原型	design.md 或现有设计规则	Figma/Stitch/HTML 生成与迭代	原型链接或可执行 HTML/代码
技术	技术规格模板、打样工程、代码规范	领域/API/数据库/时序设计	技术规格 Markdown 与 DSL
编码	需求、技术规格、代码样板	生成、修改、调试、重构	前后端代码
测试	测试规格模板、需求 AC	生成用例、API 测试、浏览器探索	测试规格、测试代码、Bug 报告
E2E	页面流程、测试用例、E2E 模板	生成并固化 Playwright	可重复运行的 TypeScript 测试
部署	环境模板、构建规则	生成和验证部署配置	YAML、构建产物、部署检查

这个工具体系的核心不是“每一步都用同一个 AI IDE”，而是保证每次转换都有清晰输入和输出，任何成员都能从仓库重新执行。

2. 需求衔接：把需求变成 AI 可执行规格

2.1 需求规格的四元结构

手册给出的可执行需求由四部分组成：

结构示意

可执行需求 = 模块切分 (增量/存量) + 字段清单 + 原型链接 + 业务规则

模块切分

先写业务背景，再按业务责任切模块；一个模块对应一个独立规格单元。必须区分：

- **增量需求：** 全新功能或新模块，需要完整定义入口、对象、流程、字段和验收；
- **存量修改：** 对已有行为做变更，需要同时写明原行为、变化点、保持不变项和兼容边界。

这项设计解决 AI 最常见的误判：把局部修改理解为推倒重做，或把全新模块误当成现有文件上的小补丁。

字段清单

字段项	作用
字段名	建立需求、UI、API、数据库之间的同名映射
类型	限定值域，减少自由文本歧义
必填性	明确缺失时应继续、拒绝还是使用默认值

字段项	作用
默认值	固定初始行为
描述	解释业务语义，而不是只解释数据类型
校验规则	形成错误路径和测试输入
UI 展示	说明用户能否看到、编辑和确认
数据来源	说明来自用户、数据库、外部服务还是模型推断

字段一旦进入规格，就应继续映射到 **schema**、类型和测试，不能在不同文档中换一套叫法。

手册使用账户字段展示格式，而不是把字段埋在段落中：

字段名	类型	必填	默认值	描述
username	string	是	—	登录用户名
email	string	是	—	电子邮箱
role	enum	否	user	用户角色
status	boolean	否	true	是否启用

这个示例还要求继续标注校验规则、UI 展示和数据来源；表格只是字段骨架，不是完整规格。

原型链接

需求必须指出对应页面、区域和交互。手册提供三种传递方式：

- 7. Figma：选中具体 **Frame**，分享并在需求中标注模块和页面；
- 8. Stitch：分享具体设计稿，不只分享项目首页；
- 9. HTML+CSS：将可交互原型提交到代码仓库，支持点击、跳转、输入、版本管理和 **Review**。

HTML 原型的优势是可直接验证，代价是需要前端能力。对于需要长期迭代的 AI 工程，原型越接近真实组件，规格到实现的信息损失越小。

业务规则

业务规则采用“条目化 + 正交分解”：

- 一条规则只表达一个独立判断；
- 每条规则拥有稳定编号；
- 每条规则可以独立构造正确输入、错误输入和边界输入；
- 规则之间避免隐藏的先后关系；存在依赖时必须显式声明；
- 验收标准引用规则编号，而不是重新改写一遍。

手册给出的规则示例包括：**R1** 用户名唯一、**R2** 邮箱格式校验、**R3** 密码不少于 8 位且包含大小写、**R4** 删除需要二次确认、**R5** 超时自动登出、**R6** 管理员不可降级。六条规则彼此独立，可以分别生成正例、反例和边界用例。

BA/产品成员也应在代码仓库中工作，使 AI 能基于当前系统事实编写新增和变更规格：需求模板和示例可以由 AI 辅助生成，但字段与业务规则必须由业务责任人确认。

2.2 原型与设计系统

路线	交付方式	优点	适合场景
AI 设计软件	Figma/Stitch URL	设计效率高，方便视觉协作	需求评审和视觉探索
HTML+CSS	仓库中的可执行代码	可交互、可版本管理、可直接演进	研发打样和高频迭代

为了避免每次生成一套新风格，手册提出 **design.md**：用 Markdown 固化颜色、字体、层级、间距和组件规则。它有两种产生方式：

- 10. 设计稿 → 提炼规则 → **design.md**;
- 11. 现有代码 → 反向解析 → **design.md**。

design.md 的本质不是增加文档数量，而是让视觉事实可被 AI 重复读取，确保新页面仍使用项目组件库和既有视觉语言。

手册同时给出 [getdesign.md](#) 作为 **design.md** 参考入口。采用外部示例时只提取颜色、字体、间距和组件规则，不直接把其他产品的视觉资产当成本项目设计。

2.3 从需求到测试的无损传递

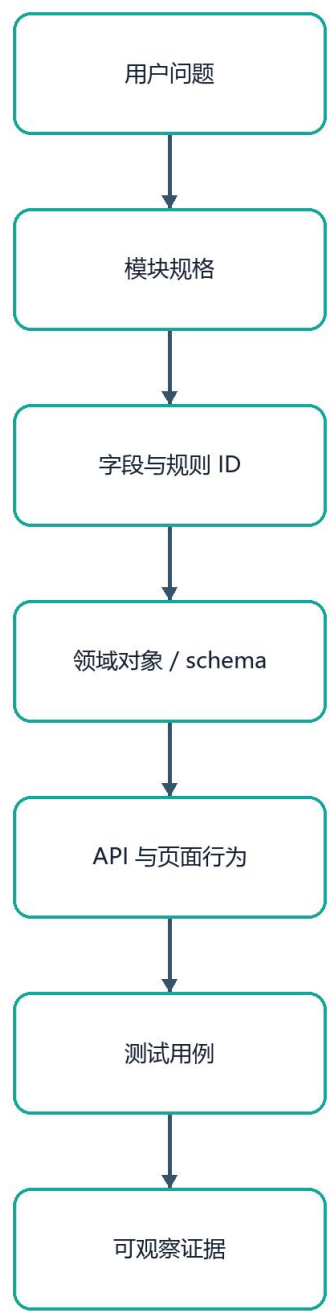


图 2 2.3 从需求到测试的无损传递（流程/架构图）

任何中间环节如果只保留“差不多的意思”，AI 就会在下一步重新猜测。团队方法要求使用稳定名称、编号和来源指针，将猜测压缩到最小。

2.4 AXIOM Space 的实践

AXIOM Space 将这一方法落实为：

- 03-用户研究与需求验证定义问题和证据；
- 04-MVP 定义明确 P0 范围和非范围；
- 05-PRD 按 Learn、Forge、Galaxy、Cognition 等用户任务定义页面和功能；
- 06-DDD-对象模型与契约将字段和规则收敛为领域对象；
- 07-TDD-验收标准定义正确路径、错误路径和完成条件；
- 08-测试计划指定测试层级、输入、输出和执行方式；
- test/ 将规格继续拆成 289 条稳定编号用例。

这些项目材料只作为方法落地证据；09 号文档的主体仍是需求规格怎样驱动 AI，而不是重复讲产品功能。

3. 开发实现：让 AI 在项目约束内持续工作

3.1 Rules、Specification、Skills

资产	它控制什么	示例	判断标准
Rules	行为边界	技术版本、命名、依赖方向、禁止事项、目录规则	是否能够明确判定违反
Specification	任务事实	用户故事、AC、字段、接口、错误、状态	是否能够导出实现和验收
Skills	执行能力	提示词、脚本、模板、CLI/API 封装	是否有明确触发、输入、输出和失败方式

规则不能写成模糊偏好，例如“代码尽量优雅”；应写成可检查要求。规格不能只写功能标题；必须描述行为、状态、输入、输出、错误和验收。Skills 也不能变成隐藏权限入口；它只能在已批准范围内复用工具。

手册用 Java 规则和注册/登录规格展示二者差异：

- Rules 可以规定 Java 17、PascalCase/camelCase、UPPER_SNAKE_CASE、单行不超过 120 字符、Egyptian 大括号风格，以及禁止 Autowired 字段注入、改用构造器注入；
- Specification 则规定用户如何注册、怎样发送验证链接或验证码、未验证账户怎样被拒绝登录、重复邮箱和错误格式怎样提示、验证失效后怎样重新发送。

前者约束所有实现的写法，后者定义某一功能的业务结果；把两者混写会导致全局规则频繁变化，或功能规则无法被单独验收。

3.2 AI 上下文体系

目录概念	职责	更新方式
standards	架构、建表、API、命名、安全等标准	当前态维护

目录概念	职责	更新方式
features	产品/BA 维护的需求规格、字段映射	当前态维护
plans	每轮任务的实现方案、任务拆分、工作状态	每轮新增，保留过程
designs	当前 DB、表、领域和接口事实，作为 Source of Truth	以最终态更新
others	架构决策、Release、测试用例等	按职责维护

其核心不是目录名称，而是区分：

- **过程方案：** 当时准备怎样做、有哪些任务、执行到哪里；采用追加方式保存；
- **事实方案：** 当前系统真实的领域、表、接口和约束；以最终态更新，作为下一轮 Plan 的输入。

AXIOM Space 采用项目化映射：

手册概念	AXIOM Space 的承载
Standards	AGENTS.md、CLAUDE.md、《开发技术规范》
Features	用户研究、MVP、PRD、TDD
Plans / 过程方案	docs/02-决策与开发过程记录/ 中的候选方案、约束和迁移过程
Designs / 事实方案	DDD 文档、Prisma schema、TypeScript 类型、Hono RPC、当前源码
Tests / Evidence	test/、scripts/test/、Playwright、测试报告

3.3 Plan/Go 规格流转

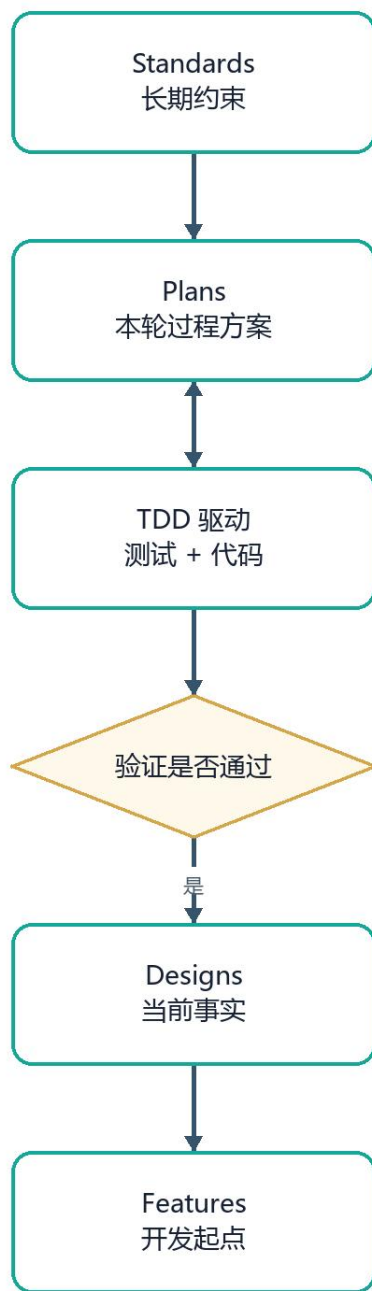


图 3 3.3 Plan/Go 规格流转 (流程/架构图)

Plan 阶段的目标不是写一份漂亮计划，而是关闭开放问题：

- 明确要改哪些对象、接口和页面；
- 明确保留哪些现有行为；
- 明确每一步的输入、输出和验证；

- 明确失败时返回哪个步骤；
- 明确哪些动作已经授权，哪些仍需人工确认；
- 记录实现状态，使任务能够分批执行、恢复和重试。

3.4 外部工具的三种接入方式

方式	工作定义	优势	适用任务
MCP	模型与数据库、API、文件系统、浏览器等外部系统通信的标准协议	接口标准化、可双向读写、配置可复用	跨工具、跨系统连接
Skills	将提示词、脚本、模板和工具封装为 AI 可调用能力单元	可复用、可团队共享、能声明专用流程	重复且需要上下文的专业任务
Commands	由 AI 或开发者直接执行 CLI 命令	最直接、可串联、易与 CI 对接	构建、测试、Git、部署、检查

MCP 解决“怎样连接”，Skills 解决“怎样正确使用”，Commands 解决“怎样直接执行”。项目 Harness 决定它们何时可用、允许访问什么、结果怎样回到 Loop。

手册中的三个最小示例分别对应：

- MCP Server 配置数据库连接，使同一外部能力可被多个 AI 工具复用；
- Commit & Push Skill 将 git add、commit、push 封装，并要求提交前检查 lint、提交信息遵循 Conventional Commits；
- 部署 Command 直接串联构建和部署命令。

这些示例的共同要求是：连接配置、流程规则 and 具体命令各归其位，不能把高权限部署命令藏在普通提示词里。

3.5 AI 编程工具分层

类别	代表工具	工作特征
CLI Agent	Claude Code、Codex CLI、OpenCode	AI 优先，适合仓库级检索、命令、长任务和自动执行
AI IDE	Cursor、Kiro、Trae	代码掌控感强，适合局部编辑和高频人工介入
增强插件/方法包	Superpower、oh-my-opencode、oh-my-codex	提供 Skills、脚本、模板和流程增强

选择标准不是品牌，而是任务：大范围理解要求读取规则和搜索引用；局部编码要求可审查 diff；长任务要求执行命令并根据失败继续；高风险任务要求权限可控、动作可追踪、结果可回退。

3.6 SDD 框架比较

维度	BMAD	Spec Kit	OpenSpec	Kiro
定位	企业级 SDD 操作系统	工程化 Spec workflow	轻量 Spec 层	IDE 原生 SDD

维度	BMAD	Spec Kit	OpenSpec	Kiro
核心	治理体系 + 多 Agent 编排	Spec 是开发入口并绑定 Git 生命周期	Spec 是持续演化的变更单元	Spec 是代码与测试的可执行源
生命周期	PRD→架构→实现全周期	与分支绑定，较短	长期存在，持续演化	开发时持续驱动
粒度	系统/模块	Feature	Change/Patch	Feature + 行为
行为表达	强规范	部分结构化	轻量 Given/When/Then	严格 EARS
流程控制	强：阶段、审批、Agent	中：Plan→Spec→Tasks	弱：自由演化	强：Spec→生成→验证
Agent 参与	多 Agent 分工	辅助执行	Prompt 级辅助	IDE 内主导执行
自动验证	依赖外部测试	依赖外部测试	较弱	内建验证较强
与代码关系	间接，通过流程	半耦合，驱动实现	弱耦合	强耦合，直接生成和校验
适合场景	大型系统、多团队、强治理	新项目 0→1	存量项目、快速迭代	小团队、高自动化
主要风险	文档和流程偏重	分支规格可能短命	自由度高，容易漂移	与 IDE 和生成链耦合

团队没有整体照搬其中任何一套，而是采用 BMAD 的治理意识、Spec Kit 的开发入口、OpenSpec 的增量演化和 Kiro 的可执行验证，最终由项目自身的文档、规则、测试和代码组成 Harness。

3.7 Superpower 的七段工程顺序

- 12. **brainstorming**: 写代码前澄清想法、比较方案、分段确认设计并保存文档；
- 13. **using-git-worktrees**: 设计通过后创建隔离分支和工作区，确认基线测试干净；
- 14. **writing-plans**: 把设计拆成约 2—5 分钟的细粒度任务，每项写出精确路径、完整动作和验证步骤；
- 15. **subagent-driven-development / executing-plans**: 按任务执行，先审规格符合性，再审代码质量；
- 16. **test-driven-development**: RED→GREEN→REFACTOR，先证明测试会失败，再写最小实现；严格执行时，测试之前写出的实现代码需要删除或按测试重新建立；
- 17. **requesting-code-review**: 任务之间按严重等级审查，关键问题阻断进度；
- 18. **finishing-a-development-branch**: 验证测试，明确合并、PR、保留或丢弃，并清理工作区。

团队采用的是这条工程顺序，而不是将 Superpower 作为项目运行依赖。这样保留方法价值，也避免外部框架限制项目的精细调整。

3.8 技术规格采用可执行 DSL

技术规格	表达标准	格式	必须产出	硬约束
领域模型	PlantUML / Smart Domain	.puml	实体、属性、关系、聚合	字段与数据库一致，实体关系明确

技术规格	表达标准	格式	必须产出	硬约束
数据库	SQL DDL / DBML / Flyway	.sql	表、索引、约束、外键	必须可执行，不能只写自然语言
API	OpenAPI 3.x	.yaml / .json	请求、响应、schema、错误	禁止只写接口名称，字段与模型一致
时序	PlantUML	.puml	调用流程和系统交互	必须反映真实调用链并包含异常分支
专题设计	Markdown	.md	权限、事务、缓存、算法等策略	只写策略与决定，不重复 API/DDDL

AXIOM Space 将同一原则映射为 Prisma schema、TypeScript 类型、Zod、Hono RPC、Mermaid 和决策 Markdown。表达格式可以变化，但“可执行、可映射、可验证”不能变化。

3.9 打样工程：让 AI 有正确作业可抄

手册提出三个打样原则：

- 19. 约定大于配置：先给 AI 一个符合团队标准的完整样板，再补充少量差异；
- 20. 编排逻辑和原子能力分离：AppService 负责用例顺序，DomainService/Repository 负责可复用能力；
- 21. 操作者和被操作对象分离：Controller/AppService 负责调度，Entity/Value Object/PO 承载业务事实。

结构示意图

```
简单 CRUD
Controller + Command → AppService → Repository → Response

复杂或有复用逻辑的 CRUD
Controller + Command → AppService → DomainService → Repository → Response

复杂查询
Controller + Query → AppService → QueryPO (或复用 PO) → Response
```

打样工程让 AI 看到请求怎样进入、规则放在哪里、数据怎样写入、响应怎样形成、异常怎样返回。高质量样板比数十条零散风格要求更容易产生一致代码。

手册的打样参考工程为 [domain-driven-design/ddd-microservices](#)。参考工程只用于研究工序和边界；进入项目的代码仍必须服从当前语言、框架、许可证和领域事实。

3.10 多任务同步开发

Git Worktree 可以把同一仓库的多个分支映射到不同文件夹，使不同 Agent 或成员在隔离工作区并行执行：

代码示例 · BASH

```
git worktree add ../feature-login feature/login
git worktree add -b feature-payment ../feature-payment
git worktree list
```

```
git worktree remove ../feature-login
```

并行开发必须满足任务边界独立、文件写集尽量不重叠、每个工作区有自己的验证结果，并在最终合并前执行跨分支集成测试。**Worktree** 是隔离机制，不是自动解决冲突的机制。

Claude Code、**Codex**、**Cursor** 等工具可以由 **Agent** 或 **subagent** 自动创建 **Worktree**，但是否并行仍由任务边界决定。多个 **Agent** 同时存在不等于可以安全并行修改同一 **schema**、同一入口或同一核心契约。

3.11 研发自测核心 Loop

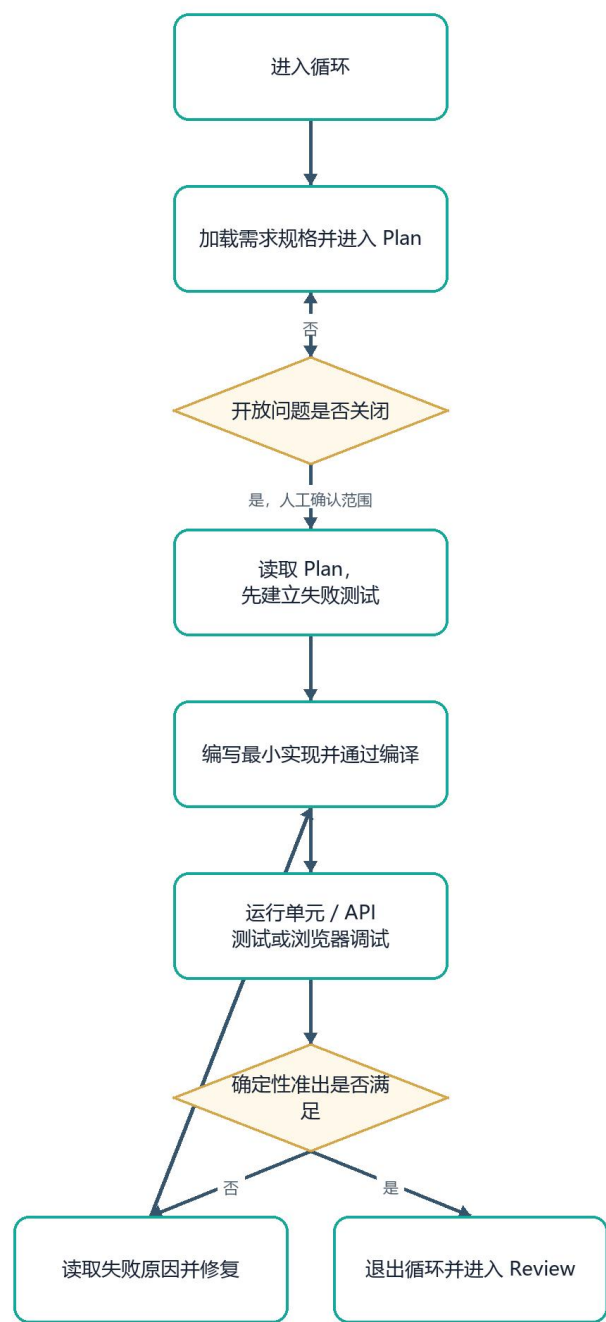


图 4 3.11 研发自测核心 Loop (流程/架构图)

以 API 任务为例：

- 22. 在任务计划中明确目标、状态和验证方式，等待确认；
- 23. 根据 Plan 编写 API 测试，先证明测试能够在缺少实现时失败；
- 24. 编写单元测试和最小实现，通过编译；

- 25. 运行 API 测试;
- 26. 修复失败测试和其他错误;
- 27. 重复执行, 直到满足准出条件。

核心思想是: **Plan** 锁定上下文并关闭开放问题; **Plan** 记录实现状态, 使任务可分批完成或重试; 验收方式必须确定, 让 **AI** 能根据失败建立自我修复标准。

手册流程图中的 **ByPass** 权限只应理解为: 团队在 **Plan** 确认后, 允许 **AI** 在本轮已批准的命令和文件范围内连续执行。它不取消破坏性操作确认、密钥保护、目录边界和最终人工审查。

核心 **Loop** 有两个运行尺度:

- 白天由成员多次发起短循环, 及时确认设计变化和跨模块影响;
 - 上下文、测试和权限已经锁定后, 长循环可以持续更久, 承担批量实现、运行、修复和重试。
- 能否长时间运行由反馈确定性决定, 不由任务“看起来简单”决定。

4. 测试验收: 用反馈闭环控制 AI 质量

4.1 AI 辅助下的分层测试策略

AI 参与开发后, 测试范围不能缩小, 反而必须更清楚地区分不同失败:

测试类型	测试对象	工具组合	人与 Agent 的责任
Lint / 静态扫描	代码风格、局部逻辑、漏洞和缺陷	TypeScript Lint、ESLint、SonarQube 等	程序员定义规则, AI 修复确定性问题
Code Review	逻辑、架构、复杂度、安全、可维护性	GitHub Copilot、Claude Code、Codex、CodeRabbit、GitHub PR	开发者 + AI Review Agent; 关键问题阻断
单元测试	函数/类行为、边界条件、异常	JUnit、pytest、Jest、Node test + AI 生成测试代码	开发者确定契约, Test Generation Agent 生成和执行用例
API 测试	schema、输入输出、鉴权、错误码、业务规则	Karate 为主、RestAssured 补充复杂逻辑 + AI 生成用例和数据	QA、后端与 API Test Agent
E2E 测试	登录、下单、支付等关键路径, 跨系统流程和前后端联动	Playwright + AI 生成脚本、selector 修复和步骤补全	QA、自动化工程师与 E2E Agent

各层测试回答的问题不同:

- **Lint** 证明基本规则没有被违反;
- 单元测试证明局部业务判断;
- **API** 测试证明边界、鉴权和系统契约;
- **E2E** 证明用户从页面上能够完成任务;
- 人工 **Review** 判断架构、产品意图和不可自动化的质量。

任何一层通过都不能替代其他层。

4.2 浏览器工具选择

手册从抽象层、速度、Token 和跨应用能力比较四类浏览器操作方式。下表中的速度与 Token 数字是团队研究时的选型量级，用于比较，不作为 AXIOM 的性能验收指标。

手册原表将 Playwright MCP 标作 “OpenWright (Playwright MCP)”。本文统一使用更直接的能力名称 “Playwright MCP”，不改变原表所指的 Accessibility Tree / DOM 自动化路线。

维度	Playwright MCP	Chrome DevTools MCP	Browser Use	Computer Use
原理	读取 Accessibility Tree / DOM 快照并按节点操作	通过 CDP 与浏览器引擎通信	Python + Playwright, AI 自主决策, 支持 DOM 与截图	截图→视觉识别→坐标/按键→沙箱执行
抽象层	Accessibility Tree	CDP Protocol	DOM + 截图	截图 + 坐标
后端	Playwright	Puppeteer + CDP	Playwright + Python	模型 Computer Use API + 沙箱
研究速度量级	快, 约 0.9 秒/步	中, 约 1.2 秒/步	中, 约 1.5 秒/步	慢, 约 0.8—2 秒/步
Token 特征	高, 结构和截图可能全传	中, 按需取数据	CLI 模式较低, 研究值约 75 token/步	高, 截图编码开销大
JS 重页面	DOM 可读时稳定	可直接取 CDP 数据	DOM 失败时可视觉兜底	依赖视觉理解
跨应用	仅浏览器	仅浏览器	仅浏览器	整个桌面

选型原则是从低成本、结构化工具开始，只有结构化信息无法证明结果时才升级到视觉：

结构示意

固定 DOM/API 可验证 → Playwright / DevTools

页面未知、流程需探索 → Browser Use

Canvas、WebGL、动效、布局 → 截图 + 视觉模型

跨浏览器与其他桌面应用 → Computer Use

4.3 从测试用例生成 E2E 的三段法

手册给出一条高价值路线：

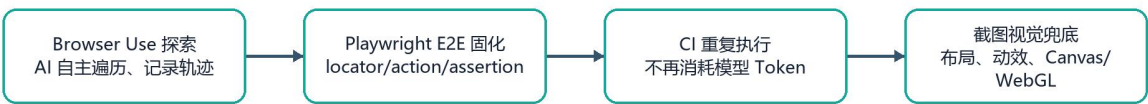


图 5 4.3 从测试用例生成 E2E 的三段法（流程/架构图）

第一段：Browser Use 探索

- 从测试用例出发，自主打开页面并完成流程；
- 记录实际操作轨迹、页面状态和失败位置；
- 发现真实可用的 locator、action 和 assertion；
- 适合页面还不稳定、选择器未知或用户流程需要发现的阶段；
- 会消耗模型 Token，不适合作为每次 CI 的唯一方式。

第二段：Playwright 固化

- 将探索结果转成 TypeScript 测试；
- 使用角色、标签、占位符或稳定测试标识定位；
- 将用户动作和可观察断言写入脚本；
- 加入 CI 后可重复运行，不需要每次调用大模型；
- 页面变化导致失败时，先判断是产品回归还是脚本需要更新。

手册把这一步压缩为四个可执行动作：

28. 要求 Browser Use 打开目标页面并完整走一遍流程；
29. 从轨迹中提取 locator、action、assertion；
30. 生成对应的 spec.ts 并加入 CI；
31. 后续使用 Playwright 重复运行，不再调用模型。

第三段：截图视觉兜底

Accessibility Tree 无法覆盖：

- 布局偏移和样式异常；
- 加载态、过渡动画和动效完成；
- Canvas、WebGL、图表或游戏画面；
- 元素可见但被其他图层遮挡。

这类场景由 Playwright 访问页面并截图，再由图像比较或视觉模型判断。视觉工具只补结构化测试的盲区，不取代结构化断言。

4.4 Playwright 脚本的最小结构

手册中的登录示例体现了稳定 E2E 的五个部分：

32. 明确起点，例如打开登录页；
33. 通过可读语义定位输入框和按钮；
34. 输入确定测试数据；

35. 执行单一用户动作；

36. 同时断言 URL 和页面可见结果。

对于非登录功能，可以先通过 API 建立测试前置状态，再进入页面验证，以减少 UI 准备步骤造成的不稳定。API 只负责准备已知状态，用户真正要验证的交互仍必须从页面执行。

一个最小脚本骨架如下：

代码示例 · TYPESCRIPT

```
import { test, expect } from '@playwright/test'

test('login success and dashboard visible', async ({ page }) => {
  await page.goto('/login')
  await page.getByPlaceholder('Email').fill('test@example.com')
  await page.getByPlaceholder('Password').fill('123456')
  await page.getByRole('button', { name: 'Login' }).click()

  await expect(page).toHaveURL(/dashboard/)
  await expect(page.getByText('Welcome')).toBeVisible()
})
```

手册还展示了“通过 API 登录并设置 session”的稳定化方向：认证本身不是本轮测试对象时，可以用 API 准备登录态；认证就是测试对象时，必须保留完整 UI 登录链路。

4.5 E2E 超级 Loop

研发自测 Loop 关注局部代码是否收敛；E2E 超级 Loop 将需求规格、界面原型和完整用户流程纳入同一个准出循环。

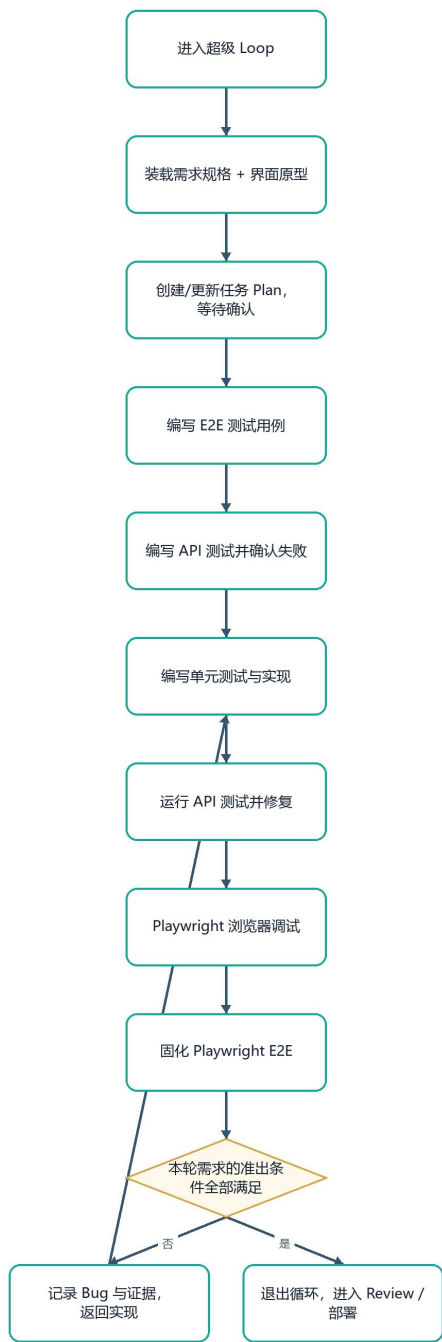


图 6 4.5 E2E 超级 Loop (流程/架构图)

超级 Loop 的规则是：

- 37. Plan 在执行前确认；
- 38. 测试用例在实现前写清；
- 39. API 测试必须能证明失败；
- 40. 实现包含单元测试并通过编译；
- 41. 失败测试必须修复，不能跳过；

- 42. 浏览器调试确认真实视觉和交互；
- 43. Playwright 覆盖本轮需求场景；
- 44. 只有全部准出要求满足，模型才退出。

4.6 AXIOM Space 的浏览器双轨实践

AXIOM 同时保留探索轨和固化轨：

轨道	实现	具体设计
探索轨	scripts/a3_browser_use.py	从场景和证明标准构造任务；设置登录状态；限制退出、删除、归档等危险操作；要求 PASS/PARTIAL/FAIL；输出 actions、visible evidence、missing evidence、bugs、next-scene readiness；保存 history、conversation、trace、截图和可选视频
固化轨	test/e2e/sdd-ui.spec.ts	固化入口、模式切换、弹窗无副作用、Forge 面板状态、遮挡检测、快捷键无领域写入、未登录入口等确定性契约
运行配置	playwright.config.ts	固定 1920×1080 Chromium；单 worker；失败保留 trace；失败截图；动作、导航和整体超时分开

这套设计直接验证了手册的关键判断：**探索和回归必须分离**。AI 负责发现真实路径，Playwright 负责把已经发现的正确路径变成低成本、可重复的工程资产。

5. Agent as Code：把 AI 工程能力纳入仓库

5.1 工作定义

Agent as Code 指将 AI 协作所需的规则、规格、能力、权限和反馈，与源代码一样版本化管理。它不是把聊天记录全部提交，而是保存下一位成员或下一种 AI 工具完成工作所需的稳定资产。

必须纳入版本治理的内容包括：

- 项目架构和依赖方向；
- 目录、命名、数据和安全规则；
- 需求、领域、API 和测试规格；
- Skills、MCP、Commands 和可重复脚本；
- AI 工具权限与配置；
- 测试入口、证据格式和失败处理；
- 关键架构决策和迁移方向。

5.2 多工具共享的文件设计

手册研究了多种方式后，选择由 AGENTS.md 建立引用关系：

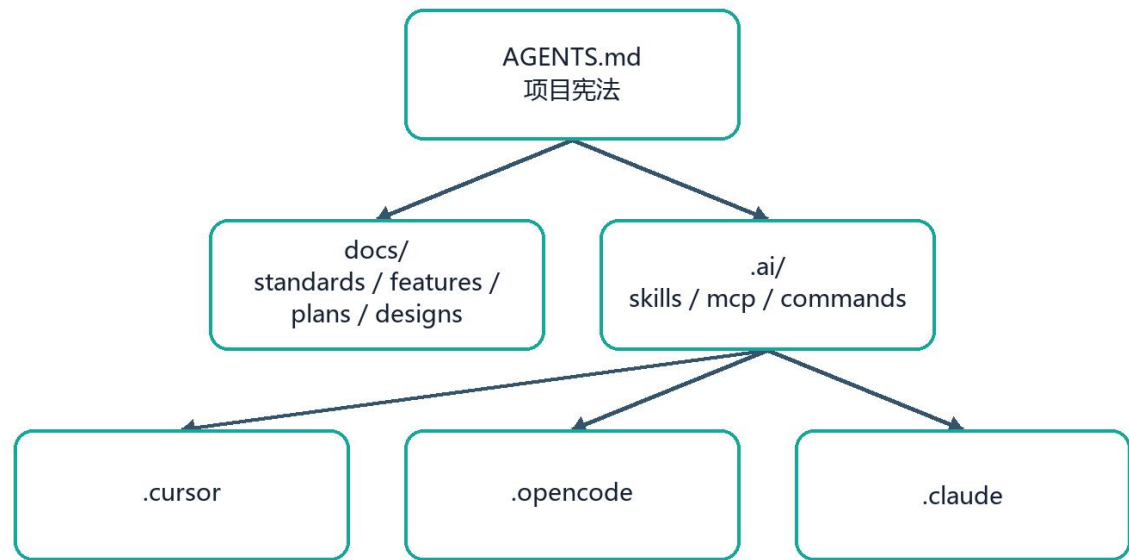


图 7 5.2 多工具共享的文件设计（流程/架构图）

其目标是一个事实、多种入口：

- AGENTS.md 只保存顶层规则和真源索引；
- docs 保存业务和技术事实；
- .ai 保存可复用执行能力；
- 各工具目录通过引用或软链接使用同一能力；
- 禁止为不同工具复制并长期维护互相漂移的规则。

5.3 AXIOM Space 的实现方式

AXIOM 当前采用双入口、同口径维护：

- AGENTS.md 供 Codex 和通用 Agent 读取；
- CLAUDE.md 供 Claude Code 读取；
- 两份规则维持相同架构、技术栈和开发边界；
- .codex/config.toml 保存 Codex 项目配置；
- .claude/settings.local.json 将命令权限限制在项目需要的范围；
- docs/00—02 保存研究、规格和过程决定；
- test/ 和 scripts/test/ 保存反馈与证据入口。

项目没有为了追求目录形式强制引入软链接。**Agent as Code** 的判断标准是规则和能力是否有明确真源、是否随代码审查、是否能被不同工具一致执行。

6. 团队实践验证与最终方法论

6.1 我们具体设计了什么

团队没有把手册停留在概念层，而是形成了六个可执行设计：

设计一：全生命周期人工/机器分工

需求研究和关键取舍由人主责；AI 负责整理、推演和生成候选；编码和测试由 AI 高频执行；验收、权限、合并和发布仍由人签发。机器参与范围扩大，但责任没有转移。

设计二：项目级上下文真源

原始问题、当前规格、过程决策、实现事实和对外文档分层保存。AI 修改前先加载项目宪法、相关规格、源码和测试，不依据文件名或旧聊天猜实现。

设计三：规格编译链

用户问题被编译为模块、字段、原型、规则，再编译为领域对象、接口、用例和断言。规格不是代码旁边的解释，而是测试和代码的上游输入。

设计四：双层 Loop

核心 Loop 负责单元/API/浏览器调试的快速收敛；E2E 超级 Loop 负责从需求到页面、Bug、Review 和部署的全链路收敛。两个 Loop 都以确定性准出条件退出。

设计五：浏览器探索—固化—视觉兜底

Browser Use 发现路径，Playwright 固化路径，截图/视觉只补结构化测试盲区。这样既利用 Agent 的探索能力，又避免每次回归都消耗模型 Token。

设计六：Agent as Code

规则、文档、工具配置、测试脚本和证据格式进入仓库。团队能力不依赖某个成员保存的提示词，也不依赖某个 AI IDE 的私有记忆。

6.2 我们具体检验了什么

检验一：规格是否真的可执行

AXIOM 将 SDD 规格拆成 289 条稳定编号用例。每条用例必须有测试方式、输入、预期输出、通过标准、失败假设和来源指针。当前复核执行：

结构示意

```
| npm test:acceptance:list
```

Total acceptance cases: 289

用例覆盖 6 条主链路、72 个核心对象、75 个细对象、聚合/服务/事件/运行时对象、6 个用户场景和 20 个 P0/P1/P2 执行入口。这个结果证明规格已经从长文档进入可被程序读取和计数的结构。

检验二：通过标准能否被编译为机器判断

test/acceptance/sdd-executable-contract.ts 会检查：

- 测试方式是否属于已支持类型；
- 来源文件和章节是否可追溯；
- 通过标准是否含状态、字段、数量、错误、来源或副作用边界；
- 失败假设是否能对应同一类判断；
- P0、Agent、UI、RAG 等不同用例是否含本领域所需契约。

因此，规格不能只写“表现正常”“体验良好”；它必须能转化为断言。

检验三：真实代码是否存在对应锚点

验收层不仅检查文档数量，还通过真实项目 adapter 检查数据库、API、领域、Agent 和 UI 锚点。规格与代码之间没有对应入口时，不能仅靠文档宣布完成。

检验四：浏览器 Agent 能否受到测试规范约束

Browser Use 执行器不是一句“帮我测一下”。它将场景、证明标准、导航边界、危险动作、停止条件和最终报告格式一起交给 Agent，并保存操作历史和可视证据。该设计检验了：浏览器 Agent 只有在任务、权限和证据三者同时明确时，才适合进入自动化流程。

检验五：探索结果能否沉淀为零模型调用回归

Playwright 测试将模式切换、弹窗副作用、面板状态、控件遮挡、快捷键和未登录入口固化为代码。后续运行由浏览器测试引擎直接完成，不依赖模型重新理解同一页面。

6.3 实践中得到的七条方法论

45. **Rule 不是终点。** Rule 只能减少越界，不能证明需求正确；必须继续连接 Spec 和测试。
46. **Spec 不是文档数量。** 能产生字段、状态、错误、来源和断言的规格才有执行价值。
47. **测试是 Agent 的反馈接口。** 没有确定性失败，AI 无法可靠自我修复；没有准出标准，循环无法正确停止。
48. **过程方案和事实方案必须分开。** 否则 AI 会把旧候选当现状，或把最终结果伪装成最初决定。
49. **探索和回归必须分开。** 用智能 Agent 发现路径，用确定脚本重复路径，用视觉模型补盲区。
50. **样板比堆规则更有效。** 一个正确的端到端打样工程能同时表达层次、命名、调用顺序和错误处理。

51. 每个项目必须建立自己的 **Harness**。开箱框架可以取用方法，但最终规则、规格、工具和证据必须服从项目事实。

6.4 哪些内容是已采用、选型参考和按需启用

层级	内容	团队结论
已采用	项目规则、SDD 文档链、DDD/TDD、稳定用例、测试 Loop、Browser Use 探索、Playwright 固化、AI 配置入库	已成为 AXIOM 工程方式
选型参考	BMAD、Spec Kit、OpenSpec、Kiro、Superpower	提取机制，不整体绑定
按需启用	Worktree 多任务并行、更多 MCP、跨 IDE 软链接、Computer Use	任务需要且边界可隔离时启用

这种分类避免两种错误：没有使用的工具不包装成既成事实；已经内化的方法也不因为未安装某个插件而被低估。

6.5 最终工作法

结构示意

研究真实问题

→ 写模块、字段、原型、规则

→ 建立领域/API/数据库/时序规格

→ 加载项目 Rules 与事实方案

→ 在 Plan 中关闭开放问题并确认权限

→ 先建立失败测试

→ AI 编写最小实现

→ 单元/API/浏览器测试

→ 失败返回实现 Loop

→ Browser Use 探索未知页面

→ Playwright 固化确定路径

→ Code Review 与人工验收

→ 更新事实、决策和证据

→ 合并与交付

这套流程把“AI 能写代码”提升为“团队能稳定地驾驭 AI 完成软件工程”。

7. AXIOM Space 的落地旁证

本章只说明手册方法在项目中对应该什么，不展开产品运行时设计。

7.1 开发阶段 AI 工具

工具或能力	在项目中的职责	治理方式
-------	---------	------

工具或能力	在项目中的职责	治理方式
OpenAI Codex	仓库检索、方案推演、补丁实现、命令验证、文档整合	读取 AGENTS.md；变更以 diff 审查；测试结果决定接受
Claude Code	代码理解、实现协作、受控命令	读取 CLAUDE.md；.claude/settings.local.json 限制命令
Browser Use	未知页面和真实用户场景探索	场景化任务、危险动作约束、证据与状态报告
Playwright	已知流程的固定 E2E 回归	固定视口、失败 trace/截图、确定性断言
MCP / Skills / Commands	外部连接、复用能力和直接执行	由项目规则、权限和任务范围决定是否可用

7.2 产品运行时 AI 能力

运行时 AI 不是本手册的主体，仅作项目能力披露：

能力	当前事实
DeepSeek	通过 OpenAI-compatible 接口作为当前运行基线
科大讯飞星火	已完成 v1.1、v2、v3.5 WebSocket 适配、HMAC-SHA256 鉴权、流式聚合、Token 用量、超时和兼容性探测，可按部署方案启用
PI Agent Core / PI AI	提供消息循环、模型抽象和工具调用底座
AXIOM Agent Harness	自研上下文装配、意图路由、工具治理、后台证据分析、资源编排和页面反馈
Ollama + BGE-M3 + Qdrant	本地语义嵌入和 Vault 作用域向量召回
LightRAG	可重建的图增强检索派生层

7.3 项目文档与方法论的对应

手册方法	AXIOM 证据
需求规格四元结构	用户研究、MVP、PRD、DDD、TDD
Rules	AGENTS.md、CLAUDE.md、开发技术规范
Specification	01-SDD 全链路
Skills / Commands	scripts/、scripts/test/、package scripts
核心 Loop	测试先行、Node 验收、API/Agent/Sidecar/UI 契约
E2E 超级 Loop	Browser Use 执行器、Playwright E2E、测试证据
Agent as Code	规则、AI 配置、文档和测试均进入仓库
过程/事实分离	02 号决策记录与当前源码/规格分开

8. 30 页 PPT 逐页研究与执行说明

本章按 PDF 文件的物理页序 1—30 逐页说明。章节分隔页也保留，因为它们定义了方法责任边界，不将其误写成独立技术。

8.1 第 1—5 页：方法论全景

页	页面主题	这一页具体设计了什么	具体怎样执行	形成的结论或资产
1	封面与版本	将研究成果命名为《团队级 AI Coding 简明手册 v0.2》，时间基线为 2026.06	把版本号、日期和研究范围作为受控基线；后续修改必须能够说明相对 v0.2 增加、删除或修正了什么	方法论不是临时聊天结论，而是可迭代的团队知识资产
2	AI 编程思想发展	原始阶段→Rule→SDD→Loop Engineering→Harness Engineering；强调每次变化是对前者的重述和扩展，不是简单推翻	先识别团队当前所处阶段，再补充失能力；Rules 不成熟时不直接谈无人值守，Spec 不可验收时不直接跑 Loop	团队目标从“半自动、有人值守”推进到“Plan 后在已批准范围内自动执行”；AXIOM 定位于 Harness 阶段
3	用问题描述处境	将模糊的“AI 开发难”拆成需求衔接、开发实现、测试验收、Agent as Code 四个问题域	对每个问题域分别建立模板、真源、执行工具和验收方式；不得用一个通用 Prompt 混合处理全部责任	形成四章方法结构，也是 09 号文档的主体结构
4	AI 赋能软件开发实现地图	将人和机器放入需求分析、技术方案、编码、研发自测、QA 测试全生命周期；区分核心 Loop 和全流程超级 Loop	人负责竞品、访谈、关键设计和签发；AI 参与文档、代码、测试和修复；失败通过 Bug 回流到实现；E2E 通过后才部署	AI Coding 被定义为全流程协作系统，不再等同于代码补全
5	AI 赋能软件开发工具体系	为需求规格、技术规格、代码、Bug 报告、E2E、部署等产物设置模板和 AI 工具入口	每个阶段明确输入模板→AI IDE/MCP/Playwright→结构化输出；输出继续作为下一阶段输入	形成可追踪的制品流水线，避免自然语言需求直接跳到生产代码

8.2 第 6—19 页：需求衔接与开发实现

页	页面主题	这一页具体设计了什么	具体怎样执行	形成的结论或资产
6	需求衔接章节页	将需求阶段独立出来，强调 BA/产品工作也是 AI 工程的一部分	在编码前先完成需求结构、字段、原型和规则；BA 也在仓库中工作，并基于当前系统事实写变更	需求不是聊天输入，而是仓库中的开发起点
7	需求规格编写	定义“模块切分（增量/存量）+字段清单+原型链接+业务规则”四元结构	先写背景；一模块一文档；增量建立新模块，存量标明变化；字段写类型/必填/默认/描述/校验/UI/来源；规则编号并独立可测	形成 AI 能理解、研发能实现、测试能追踪的需求模板
8	原型图设计	比较 Figma/Stitch URL 与 HTML+CSS；提出 design.md 统一视觉风格	需求中标出 Frame/页面映射；需要交互和版本控制时交付 HTML/代码；从设计稿提炼 design.md，或从现有代码反推设计规则	原型成为可引用、可交互、可版本管理的规格；视觉规则不靠模型临场发挥
9	开发实现章节页	将开发阶段从需求衔接中分离	需求规格通过评审后才进入 Rules、Spec、Skills、工具、打样和 Loop	防止边写需求边写实现造成上下文漂移
10	如何让 AI 听话	明确 Rules、Specification、Skills 三种资产的差异	Rules 写可判定边界；Specification 把模糊要求收敛为用户故事和 AC；Skills 将提示词、脚本和模板封装为重复能力	单一“系统提示词”被拆成行为、目标和能力三个责任层

页	页面主题	这一页具体设计了什么	具体怎样执行	形成的结论或资产
11	AI 上下文体系	设计 AGENTS.md 宪法，以及 standards、features、plans、designs、others；区分过程方案和事实方案	Features 提供起点；Standards 提供约束；Plan 保存本轮方案和状态；Designs 以最终态维护；TDD 将变更写回事实	解决长任务上下文丢失、旧方案污染当前事实和维护成本过高的问题
12	AI 调用外部工具	区分 MCP、Skills、Commands	MCP 标准连接外部系统；Skills 封装可复用操作；Commands 直接执行 CLI 并可串联；三者均受权限控制	工具接入从“模型能不能调用”升级为“用哪一层、何时调用、怎样审计”
13	AI 编程工具选择	将工具分为 CLI Agent、AI IDE、增强插件	长任务和仓库级执行用 Claude Code/Codex/OpenCode；高频局部编辑用 Cursor/Kiro/Trae；流程增强研究 Superpower 等	工具按工作形态选择，不按品牌决定方法
14	SDD 框架比较	从定位、生命周期、粒度、执行能力、行为结构、流程、Agent、验证、代码关系、场景和风险比较 BMAD、Spec Kit、OpenSpec、Kiro	先按团队规模、存量/增量、治理强度、自动验证和工具耦合做选择；不只比较文档数量	没有通用最佳框架；项目应提取合适机制并构建自己的 Harness
15	Superpower	用七个 Skills 串起 brainstorming→worktree→plan→execute→TDD→review→finish	设计先批准；隔离工作区；任务写明路径和验证；先规格审查再代码审查；RED/GREEN/REFACTOR；完成后明确分支处置	研究出一条完整 AI 开发秩序；团队吸收顺序，不把插件本身当作依赖
16	技术规格编写	规定领域模型、数据库、API、时序图、专题设计的 DSL 与硬约束	领域关系用图；数据库/API 必须可执行；时序必须对应真实调用链并含异常；专题文档只写策略和决定	技术设计由可校验语言表达，减少自然语言在实现阶段的二次猜测
17	打样工程	提出“约定大于配置、编排与原子能力分离、操作者与对象分离”，并给出三类 CQRUD 工序	选择一条正确端到端样板供 AI 复用；简单 CRUD、复杂业务和复杂查询走不同调用链；Repository/DomainService 边界固定	让 AI 通过正确代码学习架构；样板成为 Rules 的可运行证明
18	多任务同步开发	使用 Git Worktree 将同一仓库的不同分支映射到独立目录	为独立任务创建 worktree；分别初始化、测试、提交；检查列表；完成后移除；合并前做集成验证	并行建立在任务和文件隔离上，不以多个 Agent 同时写同一工作区冒充并行
19	完整核心 Loop	设计 Plan→确认→失败测试→实现→测试/浏览器调试→修复→准出的自治循环	Plan 关闭开放问题并记录状态；先创建 API/单元测试；运行失败；写最小实现；反复修复；满足确定性标准后退出	AI 自治依赖可读取的失败和明确退出条件；“生成完成”不再等于“任务完成”

8.3 第 20—27 页：测试验收与 Agent as Code

页	页面主题	这一页具体设计了什么	具体怎样执行	形成的结论或资产
20	测试验收章节页	将 QA 和研发自测从开发实现中独立出来	完成代码后仍需按测试对象、工具和负责人进入测试流程；不以 AI 参与为降低质量门槛的理由	测试是 AI 工程的反馈系统，不是展示材料
21	AI 辅助测试策略	建立 Lint、Code Review、单元、API、E2E 五层测试，并指定对象、工具和责任人	从静态规则到用户关键路径逐层执行；AI 生成/运行用例，开发者和 QA 定义契约、严重度和签发	不同层测试互补；任何一层通过都不能替代其他层

页	页面主题	这一页具体设计了什么	具体怎样执行	形成的结论或资产
22	AI 操作浏览器	比较 Playwright MCP、Chrome DevTools MCP、Browser Use、Computer Use 的原理、抽象层、引擎、速度、Token、JS 页面和跨应用能力	DOM 可读时用结构化工具；未知流 程用 Browser Use； Canvas/WebGL/布局用视觉；跨应 用才用 Computer Use	形成“优先结构化、按需视 觉、按任务权衡速度与 Token”的选择规则
23	从测试用例生 成 E2E	设计 Browser Use 探索 →Playwright 固化→截图视觉兜 底三段法	AI 走一遍流程并提取 locator/action/assertion；生成 spec.ts 加入 CI；Accessibility Tree 覆盖不到时截图识别	把高成本智能探索转成低成 本确定性回归，模型 Token 只用在不确定阶段
24	Playwright E2E 示例	用登录成功和 API 建会话展示真 实测试结构	page.goto；按 placeholder/role 定 位；填表和点击；断言 URL 与可见 内容；需要稳定前置时通过 API 建 状态	一个 E2E 必须有明确起点、 动作、断言和稳定前置，不 能只自动点击
25	E2E 超级 Loop	将需求规格、界面原型、测试用 例、API 测试、单元实现、浏览 器调试、E2E 纳入更大循环	在 Plan 后先写测试用例；实现和 API 测试循环；Playwright 调试真实 页面；固化本轮需求场景；不满足 准出就回流	Loop 从局部代码扩大到完整 用户价值，形成端到端自治 边界
26	Agent as Code 章节页	把 AI 工程文件管理设为独立能 力域	测试闭环建立后，将规则、规格、 工具和权限正式纳入仓库治理	临时有效的提示词不能算团 队能力，只有可版本化资产 才算
27	AI 工程文件管 理	用 AGENTS.md 引用 docs 和 .ai，再让 Cursor/OpenCode/Claude 等工 具共享	AGENTS.md 做宪法和索引；docs 放事实；.ai 放 Skills/MCP；工具目 录引用同一真源；避免复制漂移	实现“一个事实、多种 AI 工 具入口”的 Agent as Code

8.4 第 28—30 页：研究边界与最终判断

页	页面主题	这一页具体设计了什么	具体怎样执行	形成的结论或资产
28	附录与参考资料 章节页	将研究来源与正文方法分开	正文只保留经过团队整理的方法，原 始来源进入参考资料；重要判断能回 到来源	方法论有来源边界，避免将 外部观点伪装成项目事实
29	AI 编程实践心 得	给出三条最终经验：放弃开箱即 用幻想；每个项目建自己的 Harness；少量取用、大量实践	外部框架先拆机制，再按项目约束选 择；把有效做法写入规则、规格、脚 本和测试；持续接受反馈迭代手册	团队最终选择项目内生 Harness，而不是依赖一套 重型框架
30	参考资料	汇集 OpenAI、Martin Fowler、 Anthropic、GitHub 和 Computer Use 等研究入口	用这些来源校准 Harness、长任务、 SDD、多 Agent 和 Computer Use； 引用时区分原始论点、团队推导和项 目证据	形成 v0.2 的研究来源清单， 为后续版本继续验证和更新

9. 参考资料与采用边界

9.1 手册列出的研究入口

以下链接按《团队级 AI Coding 简明手册 v0.2》第 30 页保留，作为该版本的方法论研究入口：

- [OpenAI: Harness engineering](#)

- [Martin Fowler: Harness Engineering](#)
- [Anthropic: Harness design for long-running apps](#)
- [从 Rule、Spec 到 Harness: AI Coding 的渐进式建设路径](#)
- [让 AI 乖乖听话的几个 Rules](#)
- [Martin Fowler: Understanding Spec-Driven Development](#)
- [harness-engineering-in-ai-coding](#)
- [Attractor Guided Engineering Template](#)
- [OpenAI: Computer use guide](#)

9.2 采用边界

本文件严格区分三类结论：

52. **手册研究结论：** 五阶段演进、四个问题域、工具比较、SDD 比较、双层 Loop、Agent as Code 等；
53. **团队方法论：** 团队从研究中提炼的项目内生 Harness、规格编译链、探索—固化—视觉兜底和七条实践原则；
54. **AXIOM 项目事实：** 仓库中真实存在的规则、文档、配置、脚本、测试和模型接入。

外部工具对比不自动等于 AXIOM 已安装全部工具；项目已有代码也不自动证明所有测试在任意环境通过。具体版本、执行命令、通过数、失败数和环境前置条件统一由《04-测试说明书与测试报告》签发。

结语

《团队级 AI Coding 简明手册 v0.2》最终解决的不是“怎样提示 AI 写一段更好的代码”，而是“怎样建立一个团队可以持续使用、可以检查、可以迭代、可以承担责任的 AI 工程系统”。

AXIOM Space 将其落成一条明确工作法：

需求先成为规格，规格再成为测试；AI 在项目 Harness 中执行，失败回到 Loop，证据决定准出，团队决定交付。

— 文档结束 —